

Algorithm for File Updates in Python

Completed Portfolio Activity | Security allow list automation using Python file handling and list processing

Scenario: A health care organization maintains an allow list of IP addresses that are permitted to access a restricted subnet containing sensitive patient-related resources. The goal of this algorithm is to remove unauthorized IP addresses from the allow list by comparing the file contents against a remove list and then writing the revised list back to the file.

Project Description

This project demonstrates how Python can be used to automate a security-related file update. In the scenario, employees access restricted health care content based on whether their IP address appears in an allow list. The algorithm opens the allow list file, reads its contents, converts the data into a list, removes IP addresses that appear on a separate remove list, and writes the updated list back to the original file. This supports access control by helping ensure that only currently authorized IP addresses remain approved for the restricted subnet.

Completed Algorithm

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

remove_list = [
    "192.168.97.225",
    "192.168.158.170",
    "192.168.201.40",
    "192.168.58.57"
]

for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)

ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

The code above is a typed version of the Python algorithm. It uses file handling, string conversion, list processing, iteration, conditional logic, the `.remove()` method, the `.join()` method, and the `.write()` method to update the allow list.

Implementation Checklist

Activity Requirement	Completed Response
Open the allow list file	Stored the file name in <code>import_file</code> and opened <code>allow_list.txt</code> in read mode using <code>with open()</code> .
Read the file contents	Used <code>.read()</code> to store the current allow list contents in <code>ip_addresses</code> .
Convert the data into a list	Used <code>.split()</code> so each IP address could be checked and removed individually.
Compare against the remove list	Used a for loop to iterate through each IP address in <code>remove_list</code> .
Remove unauthorized addresses	Used an if statement and <code>.remove()</code> to delete matching IP addresses from <code>ip_addresses</code> .
Write the revised allow list	Used <code>"\n".join(ip_addresses)</code> , opened the file in write mode, and used <code>.write()</code> to save the updated list.

Detailed Section Responses

Open the file that contains the allow list

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
```

The algorithm begins by assigning the file name `allow_list.txt` to the variable `import_file`. This makes the file name easy to reuse later in the algorithm. A `with open()` statement is then used to open the file in read mode, which is appropriate because the first task is to inspect the current allow list before making changes.

Read the file contents

```
ip_addresses = file.read()
```

Inside the `with` statement, the `.read()` method reads all text from the allow list file and stores it in the variable `ip_addresses`. This gives the algorithm access to the current contents of the file so the IP addresses can be reviewed and processed.

Convert the string into a list

```
ip_addresses = ip_addresses.split()
```

The allow list must be converted from a single string into a list because individual IP addresses need to be removed. The `.split()` method separates the string wherever whitespace appears and returns a list of IP address values. After this step, `ip_addresses` can be searched and updated using list methods.

Iterate through the remove list

```
for element in remove_list:
```

The algorithm uses a `for` loop to move through every value in `remove_list`. Each value is temporarily stored in the loop variable `element`. This structure allows the same comparison process to be repeated for every IP address that should be removed.

Remove IP addresses that are on the remove list

```
if element in ip_addresses:
    ip_addresses.remove(element)
```

The conditional statement `if element in ip_addresses` checks whether the current remove-list IP address is still present in the allow list. When a match is found, `ip_addresses.remove(element)` removes that value from the list. This use of `.remove()` is appropriate because the list does not contain duplicate IP addresses, so removing the first matching value removes the unauthorized entry.

Update the file with the revised list of IP addresses

```
ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

After the unauthorized IP addresses are removed, the algorithm converts the updated list back into a string using `"\n".join(ip_addresses)`. This places each IP address on a separate line, preserving the readable format of the allow list file. The algorithm then opens the file in write mode and uses `.write()` to replace the old file contents with the revised allow list.

Security Relevance

This algorithm supports least privilege by removing IP addresses that should no longer have access to restricted content. In a health care environment, keeping an accurate allow list helps reduce the chance of unauthorized access to systems that may contain sensitive patient-related data. Automating this type of update also reduces manual effort and lowers the likelihood of leaving outdated access permissions in place.

Summary

This portfolio activity demonstrates how Python can be used to update a file that controls access to a restricted subnet. The algorithm opens the allow list file, reads its contents, and converts the IP address data from a string into a list. It then iterates through a remove list, checks whether each listed IP address appears in the allow list, and removes matching entries. Finally, it converts the updated list back into a properly formatted string and writes the revised content back to the original file. Overall, this project shows how Python file handling and list methods can support access control and routine security administration tasks.

Portfolio Notes

Skill Demonstrated	Evidence in This Document
Python file handling	Uses <code>open()</code> with read and write modes to safely access and update a text file.
Parsing file contents	Uses <code>.read()</code> and <code>.split()</code> to convert file contents into processable data.
Iteration and conditional logic	Uses a <code>for</code> loop and an <code>if</code> statement to compare remove-list values against the allow list.
Access control support	Removes unauthorized IP addresses from the allow list and writes the approved list back to the file.